QueueLess

Backend build guide

A step-by-step plan, from an empty folder structure to a production-ready API, for the QueueLess multi-tenant virtual queue platform.

Audience
The developer building `backend/`

Reference implementation
`frontend/src/lib/fakeBackend.js` (in-browser simulation)

Starting point
85 empty files and a README in `backend/`

Phases
0 to 16, in build order

Contents

0	How to use this guide	Approach, stack, how the frontend maps to the API
1	Project setup	package.json, dependencies, scripts, .env, .gitignore
2	Configuration and app shell	env.js, db.js, redis.js, app.js, server.js
3	Shared utilities and error handling	ApiError, asyncHandler, validation, constants
4	Data models	The 12 Mongoose models and their indexes
5	Authentication and roles	Signup, login, JWT, owner and staff roles
6	Tenant isolation	businessId scoping for every admin and staff read
7	Onboarding, business settings, slugs, branding	Templates, slug history, redirects, logos
8	Services and resources	CRUD with plan limits
9	Token and queue engine	Joining, numbering, calling, wait estimates
10	Real-time updates	Socket.IO rooms, events, Redis adapter
11	Staff, invites, and PIN switching	Seat limits, invite links, lockouts
12	Go live, public pages, and TV display	Checklist, public endpoints by slug
13	Plans, billing, and payments	Plan gating, checkout, webhooks, invoices
14	Notifications, analytics, and jobs	SMS, push, aggregations, scheduled tasks
15	Security, seed data, and testing	Rate limits, seeding, the 6 test suites
16	Connecting the frontend and deploying	Replacing the fake backend, going to production
A	Appendix	Endpoint list, environment variables, final checklist

Phase 0

How to use this guide

> **Goal:** understand what the backend must do, the stack it uses, and how each phase fills in the existing empty files.

Where you are starting

The `backend/` folder already contains every file the finished API needs, but each file is empty. This guide fills them in, phase by phase. Each phase lists the files it touches, what to write in them, and a "done when" checklist. Finish a phase and check its list before moving on: later phases depend on earlier ones.

The frontend already behaves like a working product by simulating the server in the browser. `frontend/src/lib/fakeBackend.js` is your specification: every function in it becomes an API endpoint or a service function here, with the same inputs, outputs, and rules. When in doubt about a business rule, read that file.

Recommended stack

Concern	Choice	Why
Runtime and framework	Node.js 18+, Express 4	Matches the frontend toolchain and the file layout (routes, controllers, middleware).
Database	MongoDB with Mongoose	The `models/` files (User.js, Business.js...) map directly to Mongoose models, and the business config is a nested document.
Cache, counters, locks	Redis (ioredis)	Atomic token numbers, PIN lockouts, rate limits, Socket.IO scaling.
Real-time	Socket.IO 4	Rooms per business, per display, per token.
Validation	Zod	The frontend already uses Zod, so schemas can be copied.
Auth	JWT (jsonwebtoken) and bcrypt	Stateless access tokens plus refresh tokens in HTTP-only cookies.
Jobs	BullMQ (on Redis) or node-cron	No-show timeouts, cleanup, reminders.
Payments	Razorpay (INR) or Stripe	Prices in the product are in rupees.

SMS and email	MSG91 or Twilio for SMS; Resend, SES, or Nodemailer for email	Alerts and staff invites.
File storage	Cloudinary or S3	Business logos (the frontend stores them as data URLs today).
Testing	Jest, Supertest, mongodb-memory-server	Fast, isolated API tests.

The request path

Every HTTP request follows the same path through the folders. Keep each layer to its job:

```
request
  -> routes/*.routes.js         URL + HTTP method, list of middleware
  -> middleware/*.middleware.js auth, role, tenant, validation, plan limits
  -> controllers/*.controller.js read req, call a service, send the response
  -> services/*.service.js      business rules (no req/res here)
  -> models/*.js                database reads and writes
  -> sockets/*.socket.js        broadcast the change to open screens
```

Controllers stay thin. Services hold the rules so jobs, sockets, and tests can reuse them without HTTP.

How the frontend maps to the API

fakeBackend.js function	Becomes
`signup`, `login`, `logout`	`POST /api/auth/signup`, `/login`, `/logout`
`completeWelcome`, `changePlan`	`POST /api/onboarding/welcome`, `POST /api/billing/checkout`
`saveService`, `saveResource`, `delete*`	`/api/admin/services`, `/api/admin/resources` (CRUD)
`joinQueue`, `leaveQueue`, `rateToken`	`POST /api/public/:slug/tokens`, `/tokens/:id/leave`, `/rate`
`callNext`, `markDone`, `markNoShow`, `recall`	`POST /api/staff/resources/:id/call-next`, `/api/staff/tokens/:id/…`
`createInvite`, `resendInvite`, `revokeInvite`, `acceptInvite`	`/api/admin/invites`, `/api/invites/:token/accept`
`switchUserWithPin`	`POST /api/staff/switch-user`
`checkSlugAvailability`, `changeSlug`, `goLive`	`/api/business/slug/check`, `PATCH /api/business/slug`, `POST /api/business/go-live`
`updateBranding`, `saveSettings`	`PATCH /api/business/branding`, `PATCH /api/business/settings`
`resolveSlug`	`GET /api/public/:slug` (returns business or redirect)
`simulateTick`	Not needed: real customers and staff replace it.

The full endpoint list is in the Appendix.

Phase 1 Project setup

> **Goal:** a backend project that installs, starts, and reloads on change.
> **Files:** `package.json, .env.example, .gitignore`

Step 1.1: Fill in package.json

The file exists but is empty, so run `npm init` only if you prefer; otherwise write it by hand. Use ES modules to match the frontend.

```
{
  "name": "queueless-backend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "main": "server.js",
  "engines": { "node": ">=18" },
  "scripts": {
    "dev": "nodemon server.js",
    "start": "node server.js",
    "seed": "node src/seed/seed.js",
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand",
    "lint": "eslint ."
  }
}
```

Step 1.2: Install dependencies

```
cd backend
npm install express mongoose ioredis socket.io @socket.io/redis-adapter \
  jsonwebtoken bcrypt zod cookie-parser cors helmet compression morgan \
  express-rate-limit rate-limit-redis bullmq nanoid dotenv multer \
  razorpay qrcode

npm install -D nodemon jest supertest mongodb-memory-server eslint
```

Swap `razorpay` for `stripe`, and add your SMS and email SDKs, once you choose providers (Phase 13 and 14).

Step 1.3: .env.example

List every variable with a safe placeholder. Never put real secrets in this file; copy it to `.env` locally.

```
NODE_ENV=development
PORT=4000
CLIENT_URL=http://localhost:5173
MONGO_URI=mongodb://localhost:27017/queueless
REDIS_URL=redis://localhost:6379
JWT_ACCESS_SECRET=change-me
JWT_REFRESH_SECRET=change-me-too
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=30d
INVITE_EXPIRY_DAYS=7
SLUG_REDIRECT_DAYS=90
RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=
SMS_PROVIDER_KEY=
EMAIL_FROM=QueueLess <no-reply@queueless.app>
EMAIL_API_KEY=
CLOUDINARY_URL=
```

Step 1.4: .gitignore

```
node_modules/
.env
coverage/
logs/
*.log
```

Step 1.5: Local services

Run MongoDB and Redis locally, most easily with Docker:

```
docker run -d --name ql-mongo -p 27017:27017 mongo:7
docker run -d --name ql-redis -p 6379:6379 redis:7
```

MongoDB must run as a replica set if you use multi-document transactions (Phase 9). For local work: `docker run -d -p 27017:27017 mongo:7 --replSet rs0`, then run `rs.initiate()` once in mongosh.

Done when

- ☐ `npm install` finishes without errors
- ☐ `.env` exists locally and is ignored by git
- ☐ MongoDB and Redis are reachable on their default ports

Phase 2 Configuration and app shell

> **Goal:** the server boots, connects to MongoDB and Redis, and answers a health check.
> **Files:** `src/config/env.js, db.js, redis.js, socket.js, app.js, server.js`

Step 2.1: src/config/env.js

Load `.env` with dotenv and validate it with Zod, so the server refuses to start with missing or malformed settings. Export one frozen `env` object; nothing else should read `process.env` directly.

```
import 'dotenv/config';
import { z } from 'zod';

const schema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().default(4000),
  CLIENT_URL: z.string().url(),
  MONGO_URI: z.string().min(1),
  REDIS_URL: z.string().min(1),
  JWT_ACCESS_SECRET: z.string().min(16),
  JWT_REFRESH_SECRET: z.string().min(16),
  // ...the rest of .env.example
});

export const env = Object.freeze(schema.parse(process.env));
```

Step 2.2: src/config/db.js and redis.js

- `db.js` exports `connectDB()`, which calls `mongoose.connect(env.MONGO_URI)`, logs success, and exits the process on failure. It also exports `disconnectDB()` for tests and graceful shutdown.
- `redis.js` exports a shared ioredis client, plus a `duplicate()` pair (publisher and subscriber) for the Socket.IO adapter in Phase 10.

Step 2.3: app.js

Build and export the Express app without starting it, so tests can import it.

1. Security and parsing: `helmet()`, `cors({ origin: env.CLIENT_URL, credentials: true })`, `express.json({ limit: '1mb' })`, `cookieParser()`, `compression()`, and `morgan` outside tests.
2. `GET /api/health` returns `{ ok: true }`.
3. Mount route files under `/api` (added phase by phase).
4. A 404 handler, then `error.middleware.js` last (Phase 3).

Step 2.4: server.js

1. Import `env`, call `connectDB()`, create an HTTP server from `app`.
2. Attach Socket.IO through `src/sockets/index.js` (stub it for now; it is filled in Phase 10).
3. Start background job workers (Phase 14).

4. Listen on `env.PORT`. On `SIGTERM` and `SIGINT`, stop accepting connections, close sockets, Redis, and MongoDB, then exit.

`src/config/socket.js` holds Socket.IO options (CORS origin, ping intervals, path) so `sockets/index.js` stays focused on events.

Done when

- ☐ `npm run dev` starts and logs the database and Redis connections
- ☐ `curl localhost:4000/api/health` returns `{"ok":true}`
- ☐ Removing a required variable from `.env` stops the server with a clear message

Phase 3 Shared utilities and error handling

> **Goal:** one consistent error format and one way to validate input, used by every later phase.
> **Files:** `src/utils/ApiError.js, asyncHandler.js, constants.js, src/middleware/error.middleware.js, validate.middleware.js`

Step 3.1: utils/ApiError.js

A class extending `Error` with `statusCode`, a machine-readable `code`, and optional `details`. Add static helpers so services can throw readable errors:

```
throw ApiError.badRequest('INVALID_SLUG', 'Use lowercase letters, numbers, and hyphens');
throw ApiError.forbidden('PLAN_LIMIT', 'Upgrade to add more counters', { limitKey: 'counters',
used, limit });
throw ApiError.notFound('TOKEN_NOT_FOUND');
```

Use the same error codes the frontend already reacts to: `not_live`, `closed`, `limit`, `seat_limit`, `wrong_pin`, `locked`, `invalid`.

Step 3.2: utils/asyncHandler.js

Wraps async controllers so rejected promises reach the error middleware: `(fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next)`.

Step 3.3: middleware/error.middleware.js

The last middleware in `app.js`. It converts errors into one response shape:

```
{ "ok": false, "error": { "code": "PLAN_LIMIT", "message": "Upgrade to add more counters",
"details": {...} } }
```

- `ApiError`: use its status and code.

- Zod errors: 400 with field-level messages in `details`.
- Mongo duplicate key (11000): 409 `CONFLICT`.
- Anything else: 500 with a generic message; log the stack, never send it in production.

Successful responses use `{ "ok": true, ...data }` to match the fake backend's return values.

Step 3.4: middleware/validate.middleware.js

`validate({ body, params, query })` takes Zod schemas, parses the request parts, replaces them with the parsed values, and passes Zod errors to the error middleware.

Step 3.5: utils/constants.js

Copy shared values from `frontend/src/config/constants.js` and `plans.js`: token statuses, active statuses, queue modes, reserved slugs, almost-turn threshold (3), no-show minutes (5), invite expiry (7 days), slug redirect (90 days), PIN attempts (5) and lock (30 seconds), and the four plans with limits and features.

> Keep plans in one place. A later improvement is a small shared package used by both frontend and backend, so prices and limits can never drift apart.

Done when

- ☐ A test route that throws `ApiError.notFound()` returns a 404 in the standard shape
- ☐ An invalid body on a validated route returns 400 with field messages
- ☐ An unexpected error returns 500 without leaking the stack

Phase 4 Data models

> **Goal:** all 12 Mongoose models, with the fields the frontend already uses and the indexes the queries need.
> **Files:** `src/models/*.js`

Mirror the shapes in `frontend/src/data/`. Every model owned by a business gets a `businessId` field and an index that starts with `businessId`. Add `timestamps: true` to every schema.

Model	Key fields	Indexes
User	name, email (lowercase), passwordHash, role (owner, manager, staff), businessId, resourceId, pinHash, status (active, deactivated), lastLoginAt, avatarColor, refreshTokenVersion	email unique; businessId + role

Business	name, slug, type, plan, billingCycle, isLive, isOpen, contact, dailyCustomers, labels {resource, resourcePlural, service}, branding {logoUrl, brandColor, welcomeMessage, hidePoweredBy}, settings {queueMode, tokenPrefix, allowPriority, allowAnyAvailable, maxTokensPerDay, noShowMinutes, customerFields[]}, hours, notifications, setup flags, smsUsed, ownerId	slug unique
Service	businessId, name, avgMinutes, prefix, isActive, sortOrder	businessId
Resource	businessId, name, serviceIds[], isActive, isPaused	businessId
Queue	businessId, queueKey (shared, svc:<id>, res:<id>, res:any), date (YYYY-MM-DD), counters per prefix, isPaused	businessId + queueKey + date unique
Token	businessId, queueKey, number, code, serviceId, resourceId, customer {name, phone, extra fields}, priority, status, source (qr, walk-in), createdAt, calledAt, servedAt, doneAt, cancelledAt, cancelledBy, sortAt, recalls, rating, guestSecret	businessId + status + queueKey + priority + sortAt; businessId + createdAt; businessId + customer.phone + status
Subscription	businessId, plan, cycle, status (trialing, active, past_due, cancelled), providerSubscriptionId, currentPeriodEnd, trialEndsAt	businessId unique
Invoice	businessId, number, date, plan, amount (paise), status, providerInvoiceId, pdfUrl	businessId + date
StaffInvite	businessId, email, role, resourceId, tokenHash, status (pending, accepted, revoked), expiresAt, invitedBy, acceptedAt, userId	tokenHash unique; businessId + email
SlugHistory	businessId, slug, redirectUntil	slug unique; TTL on redirectUntil
ActivityLog	businessId, actorId, kind (call, join, settings, invite, staff, billing, rating), text, meta	businessId + createdAt; optional TTL (for example 180 days)
EnterpriseLead	name, email, company, locations, message, status	createdAt

Notes on specific models

- **Business settings stay embedded.** Labels, branding, settings, hours, and setup flags are always read with the business, so keep them as subdocuments. Services and resources are separate collections because they are listed, filtered, and limited by plan.
- **Customer fields are dynamic.** Store the token's customer as `{ name, phone, ...extra }` with `strict: false` on that subdocument, and validate it at request time against the business's `customerFields` (Phase 9).
- **Never store secrets in plain text.** Passwords and PINs are bcrypt hashes; invite tokens are stored as SHA-256 hashes so a database leak does not expose working links.

- **Strip private fields.** Add a `toJSON` transform that removes `passwordHash`, `pinHash`, `tokenHash`, and `__v`.
- **Reserved slugs.** Validate slugs in `utils/slug.js` (Phase 7), not only in the schema.

Done when

- ☐ All 12 models compile and indexes are created on startup (`autoIndex` in development)
- ☐ Creating two businesses with the same slug fails with a duplicate-key error
- ☐ `JSON.stringify(user)` never contains a hash

Phase 5 Authentication and roles

> **Goal:** owners and staff log in at one endpoint and receive tokens that carry their role and business.
> **Files:** `routes/auth.routes.js, controllers/auth.controller.js, validators/auth.validator.js, middleware/auth.middleware.js, role.middleware.js, tests/auth.test.js`

Step 5.1: validators/auth.validator.js

Copy `signupSchema` and `loginSchema` from `frontend/src/lib/validators.js`: full name, work email, password of at least 8 characters, business name, business type, terms accepted.

Step 5.2: Signup (POST /api/auth/signup)

One request creates both the owner and the business, inside a MongoDB transaction:

1. Reject if the email exists (409, "An account with this email already exists").
2. Build the business from the type's template (same logic as `frontend/src/config/templates.js`), with a unique slug from `slug.service.uniqueSlug(name)`, plan `free`, `isLive: false`.
3. Insert the template's services and resources.
4. Create the owner with a bcrypt password hash (cost 12) and `businessId`.
5. Log "Business created from the <type> template" in ActivityLog.
6. Issue tokens (Step 5.4) and return `{ user, business }`. Save `?plan=` as `intendedPlan` for the plan page.

Step 5.3: Login (POST /api/auth/login)

- Find the user by lowercase email and compare with bcrypt. Return the same message for "no user" and "wrong password".
- Reject deactivated users with 403 and the message the frontend shows.
- Update `lastLoginAt`, issue tokens, and return `{ user, home }` where `home` is `/admin` for owners and `/staff` for staff and managers.

Step 5.4: Tokens

- **Access token:** a JWT signed with `JWT_ACCESS_SECRET`, valid 15 minutes, payload `{ sub: userId, businessId, role }`. Send it in the response body; the frontend keeps it in memory.
- **Refresh token:** a JWT valid 30 days with `{ sub, v: refreshTokenVersion }`, in an HTTP-only, Secure, SameSite=Lax cookie scoped to `/api/auth`.
- `POST /api/auth/refresh` verifies the cookie and the version, then issues a new pair.
- `POST /api/auth/logout` clears the cookie and increments `refreshTokenVersion` (logs out every device).
- `GET /api/auth/me` returns the user and their business for app start-up.

Step 5.5: auth.middleware.js and role.middleware.js

- `requireAuth` reads `Authorization: Bearer`, verifies the JWT, loads the user, rejects if missing or deactivated, and sets `req.user` and `req.businessId`.
- `requireRole('owner')` and `requireRole('staff', 'manager')` compare `req.user.role` and return 403 otherwise.

Step 5.6: Forgot password

`POST /api/auth/forgot-password` always returns success (so it never reveals whether an email exists) and emails a single-use link valid for 1 hour. `POST /api/auth/reset-password` sets the new password and increments `refreshTokenVersion`.

Step 5.7: tests/auth.test.js

Cover: signup creates a user and a business with a unique slug; duplicate email returns 409; login with a wrong password returns 401; a deactivated user cannot log in; an expired access token returns 401; refresh issues new tokens; logout invalidates refresh.

Done when

- ☐ Signup, login, refresh, logout, and me work with Supertest
- ☐ Owner-only routes reject staff tokens with 403
- ☐ `tests/auth.test.js` passes

Phase 6 Tenant isolation

> **Goal:** make it impossible for one business to read or change another business's data.
> **Files:** `middleware/tenant.middleware.js`, `services/tenant.service.js`, `tests/tenantIsolation.test.js`

QueueLess is one app shared by every business. Isolation is the most important security property in the backend, so build it before any business features.

The two ways a request finds its business

Who	How the business is found	Where
Owner, manager, staff	`businessId` from the verified JWT. Never from the URL or the body.	All `/api/admin/*`, `/api/staff/*`, `/api/business/*`
Customer, TV display	`slug` in the URL, resolved to a business (or a redirect).	`/api/public/:slug/*`, `/api/display/:slug`

Step 6.1: tenant.middleware.js

- `tenantScope` (after `requireAuth`) loads the business by `req.user.businessId`, returns 404 if it was deleted, and sets `req.business`.
- If a request includes a `?business=` slug or a body `businessId` that differs from the session, return 403 `ACCESS_DENIED`. This matches the frontend's "Access denied" screen.
- `publicBusiness` (for public routes) resolves `:slug`. If the slug is in SlugHistory and still within its redirect window, respond `308` with `{ redirectTo: newSlug }`. Reserved or unknown slugs return 404.

Step 6.2: tenant.service.js

Provide scoped helpers so services cannot forget the filter:

```
export const scoped = (Model, businessId) => ({
  find: (filter = {}, ...rest) => Model.find({ ...filter, businessId }, ...rest),
  findOne: (filter = {}, ...rest) => Model.findOne({ ...filter, businessId }, ...rest),
  updateOne: (filter, update) => Model.updateOne({ ...filter, businessId }, update),
  deleteMany: (filter) => Model.deleteMany({ ...filter, businessId }),
  create: (doc) => Model.create({ ...doc, businessId }),
});
```

Rule: every query against a business-owned model goes through `scoped()` or includes `businessId` explicitly. Look up records by `{ _id, businessId }`, never `_id` alone, so guessing another business's id returns 404.

Staff have a second limit: a staff user (not a manager) may act only on their assigned `resourceId`. Enforce this in the staff service functions.

Step 6.3: tests/tenantIsolation.test.js

Seed two businesses, A and B, each with an owner and a staff member. Then assert that A's tokens cannot:

- list B's services, resources, staff, tokens, invites, or invoices;
- update or delete a B record by id (expect 404, not 403, so ids are not confirmed);
- call next on a B resource;
- pass `?business=b-slug` (expect 403);
- as staff, act on a resource in their own business that is not assigned to them (expect 403).

Done when

- ☐ Every authenticated route uses `requireAuth`, `tenantScope`, and the right `requireRole`
- ☐ `tests/tenantIsolation.test.js` passes and stays in CI permanently

Phase 7 Onboarding, business settings, slugs, and branding

> **Goal:** owners can finish onboarding and change everything in Settings, for free.
> **Files:** `onboarding.routes/controller, business.routes/controller, validators/business.validator.js, services/slug.service.js, branding.service.js, utils/slug.js, tests/slug.test.js`

Step 7.1: Onboarding

- `POST /api/onboarding/welcome` with `{ type, dailyCustomers }`. If the type changed and the business has no tokens yet, re-seed labels, settings, services, and resources from the new template (in a transaction). Otherwise only save `dailyCustomers`.
- "Skip for now" and the Free plan both call `POST /api/billing/plan` with `free` (Phase 13).

Step 7.2: Business settings

`GET /api/business` returns the full business. `PATCH /api/business/settings` accepts any of: name, type, contact, labels, settings, hours, notifications. Validate each part with Zod:

- **Queue rules:** queueMode is shared, per_service, or per_resource; tokenPrefix is 1 to 2 capital letters; maxTokensPerDay at least 1; noShowMinutes 1 to 30.
- **Join form:** `name` and `phone` must always be present and required; keys are unique; select fields need at least one option.
- **Hours:** seven days with `open`, `from`, `to` in HH:MM, and `from` earlier than `to`.
- Changing queue mode while tokens are active is allowed but return `{ warning: 'ACTIVE_TOKENS', count }` unless the request includes `confirm: true`, matching the frontend dialog.

- Set setup flags as the frontend does: join form changed sets `joinFormCustomised`; hours changed sets `hoursSet`.

> Plans never gate customisation. Only the paid extras (hiding "Powered by QueueLess", priority taking effect, SMS) check the plan; saving the setting itself is always allowed.

Step 7.3: Slugs (utils/slug.js and slug.service.js)

- `utils/slug.js`: `slugify(text)` and `slugProblem(slug)` (3 to 40 characters, `^[a-z0-9](?:[a-z0-9-]{1,38}[a-z0-9])?$`, not reserved). Copy from the frontend.
- `checkAvailability(slug, businessId)` returns `{ available, reason, current }`. A slug is taken if another business uses it or holds it in active SlugHistory.
- `uniqueSlug(base)` appends -2, -3... until free.
- `changeSlug(businessId, newSlug)` in a transaction: if the business is live, insert the old slug into SlugHistory with `redirectUntil = now + 90 days`; if the new slug is in this business's own history, delete that entry; update the business; set `slugConfirmed`; log activity.
- Endpoints: `GET /api/business/slug/check?slug=` and `PATCH /api/business/slug`. When live, require `confirmBreakingQr: true` in the body (the checkbox in the dialog).
- A TTL index on `SlugHistory.redirectUntil` removes expired redirects automatically.

Step 7.4: Branding (branding.service.js)

- `PATCH /api/business/branding` accepts brandColor (hex), welcomeMessage (at most 120 characters), and hidePoweredBy. Store hidePoweredBy as sent, but public responses only honour it when the plan has `removeBranding`.
- `POST /api/business/logo`: multipart upload through multer (memory storage, images only, at most 400 KB), uploaded to Cloudinary or S3, URL saved in `branding.logoUrl`. `DELETE /api/business/logo` removes it.
- Mark `setup.brandingSet` on save.

Step 7.5: tests/slug.test.js

Cover: reserved and malformed slugs are rejected; changing a slug before go live leaves no history; after go live the old slug redirects with 308; a slug in another business's history is unavailable; the TTL expiry makes it available again (simulate by moving `redirectUntil` into the past).

Done when

- ☐ Every Settings tab in the frontend has a working endpoint
- ☐ Old slugs redirect for 90 days, then 404
- ☐ `tests/slug.test.js` passes

Phase 8 Services and resources

> **Goal:** owners manage what customers queue for and who serves them, within plan limits.
> **Files:** `admin.routes.js, admin.controller.js, middleware/planLimit.middleware.js`

Step 8.1: Services

Endpoint	Behaviour
`GET /api/admin/services`	List, with optional `?status=active|inactive&q=`, plus a waiting count per service.
`POST /api/admin/services`	Body: name, avgMinutes (1 to 240), prefix (1 to 2 capitals), handledBy (resource ids), isActive. Adds the service id to each chosen resource.
`PATCH /api/admin/services/:id`	Same fields. Updates resource membership but never leaves a resource with zero services.
`POST /api/admin/services/bulk`	`{ ids, action: 'activate'|'deactivate'|'delete' }`.
`DELETE /api/admin/services/:id`	Refuse if it is the last service ("Keep at least one service so customers can join"). Past tokens keep their `serviceId`.

Step 8.2: Resources (doctors, counters, stylists)

The same pattern under `/api/admin/resources`, with name, serviceIds (at least one), and isActive. The list includes assigned staff names, the current token, and the waiting count. Deleting the last resource is refused.

Step 8.3: planLimit.middleware.js

`enforceLimit('counters')` counts active resources against the plan's limit and throws 403 `PLAN_LIMIT` with `{ limitKey, used, limit }`, which the frontend uses for its upgrade prompt. Apply it when a resource is created active, or switched from inactive to active (including bulk).

> Limits only block new additions. Never delete or deactivate existing data when a business is over its limit, for example after a downgrade.

Done when

- ☐ A Free business with 1 active counter gets `PLAN_LIMIT` when adding a second
- ☐ The same business can still edit and rename its existing counters
- ☐ Deleting the last service or resource returns a clear 400

Phase 9 Token and queue engine

> **Goal:** the core product: customers join, staff call, every number stays correct under load.
> **Files:** `token.routes/controller, staff.routes/controller, services/token.service.js, queue.service.js, eta.service.js, utils/tokenNumber.js, validators/token.validator.js, tests/token.test.js, tests/queue.test.js`

Step 9.1: Queue keys (queue.service.js)

Port `frontend/src/lib/queueMath.js`. The queue key decides which line a token is in:

- **shared**: one line, key `shared`.
- **per_service**: key `svc:<serviceId>`. A resource can call tokens whose service it handles.
- **per_resource**: key `res:<resourceId>`, or `res:any` for "Any available", which any resource handling that service can call.

Order inside a line: priority first, then `sortAt ?? createdAt`. "Skip to end" sets `sortAt = now` and clears priority.

Step 9.2: Token numbers (utils/tokenNumber.js)

Numbers must never repeat, even when two customers join in the same millisecond. Use an atomic counter per business, prefix, and day:

```
// Redis: INCR is atomic; expire the key after the day ends
const key = `tok:${businessId}:${dateKey}:${prefix}`;
const number = await redis.incr(key);
if (number === 1) await redis.expire(key, 60 * 60 * 36);
const code = `${prefix}-${String(number).padStart(3, '0')}`;
```

Alternative without Redis: `Queue.findOneAndUpdate({ businessId, queueKey, date }, { $inc: { [`counters.${prefix}`]: 1 } }, { upsert: true, new: true })`. Use the business's time zone (default Asia/Kolkata) for `dateKey`.

Step 9.3: Joining (POST /api/public/:slug/tokens)

Body: `{ serviceId?, resourceId? | 'any', customer: {...}, priority? }`. In order:

1. Resolve the business by slug (Phase 6). Reject `not_live` if not live, `closed` if paused.
2. Validate `customer` against the business's `customerFields` with a schema built at runtime (port `buildJoinSchema`). Name and a 10-digit Indian mobile number are always required.
3. If an active token (WAITING, CALLED, SERVING) already exists for this phone, return it with `existing: true` instead of creating a duplicate.
4. Check the daily limit: the smaller of `settings.maxTokensPerDay` and the plan's `tokensPerDay`. Reject with `limit`.

5. Pick the service (given, or the resource's first, or the business's first) and prefix (shared uses `tokenPrefix`; other modes use the service prefix).
6. Keep `priority` only if `allowPriority` is on and the plan has `priorityQueues`.
7. Get the number (Step 9.2), create the token with a random `guestSecret`, set `setup.testTokenRun`, and broadcast `token:created` (Phase 10).
8. Return `{ token, guestSecret }`. The customer page stores both; the secret is required to leave the line or rate a visit, so a token id alone cannot cancel someone's place.

Protect this endpoint with a rate limit per IP and per phone number (Phase 15).

Step 9.4: Token status (GET /api/public/:slug/tokens/:id)

Return the token plus computed fields: `ahead` (WAITING tokens before it in its line), `waitMinutes` (from eta.service), `progress`, `nowServing` for its line, and the resource name when called. Return 404 if the token is not in this business.

eta.service.js: `round((ahead + 0.5) × service.avgMinutes ÷ activeServers)`, where activeServers is 1 for a single resource line, the number of active, unpaused resources handling the service for per-service lines, and all active resources for shared lines. Later, replace `avgMinutes` with a rolling average of real service times from the last 7 days.

Step 9.5: Staff actions (staff.routes.js)

Endpoint	Effect
`POST /api/staff/resources/:id/call-next`	Finish the resource's current token (DONE), take the first waiting token it can serve, set CALLED with `resourceId` and `calledAt`.
`POST /api/staff/tokens/:id/call`	Call a specific waiting token ("Call this one").
`POST /api/staff/tokens/:id/arrived`	CALLED to SERVING, set `servedAt`.
`POST /api/staff/tokens/:id/done`, `/no-show`	Set DONE or NO_SHOW with `doneAt`.
`POST /api/staff/tokens/:id/recall`	Reset `calledAt`, increment `recalls`.
`POST /api/staff/tokens/:id/skip`, `/remove`	Move to the end, or CANCELLED by staff.
`POST /api/staff/walk-in`	Same as joining, with `source: 'walk-in'`.
`PATCH /api/staff/resources/:id/pause`	Pause or resume one resource.
`GET /api/staff/console`	Resources the user may use (own only for staff, all for managers), current tokens, waiting lists, today's metrics.

> **Race condition to prevent:** two staff pressing "Call next" at the same moment must never call the same customer. Claim the token atomically: `Token.findOneAndUpdate({ _id, status: 'WAITING' }, {`

`$set: { status: 'CALLED', ... } })`. If it returns null, someone else took it, so try the next token. Validate every status change against an allowed-transitions table and return 409 on invalid moves.

Step 9.6: Customer actions

`POST /api/public/:slug/tokens/:id/leave` (WAITING only, with `guestSecret`) and `POST /api/public/:slug/tokens/:id/rate` (DONE only, rating 1 to 5, once).

Step 9.7: tests/token.test.js and tests/queue.test.js

- **token.test.js:** numbering increments per prefix and resets per day; 20 concurrent joins produce 20 unique numbers; duplicate phone returns the existing token; daily limit, closed, and not-live errors; priority ignored on Free; dynamic field validation; leave requires the secret.
- **queue.test.js:** ordering by priority then arrival; each queue mode routes calls correctly; "any available" tokens are callable by eligible resources; concurrent call-next never double-calls; skip moves to the end; invalid transitions return 409; wait estimates match the formula.

Done when

- ☐ A customer can join, see their position, be called, and finish, using only HTTP
- ☐ Concurrency tests pass reliably
- ☐ `tests/token.test.js` and `tests/queue.test.js` pass

Phase 10 Real-time updates

Goal: every open screen (customer phone, staff console, admin queues, TV) updates within a second of any change.
Files: `src/sockets/index.js, auth.socket.js, queue.socket.js, display.socket.js, src/config/socket.js`

Step 10.1: sockets/index.js

1. Create the Socket.IO server with options from `config/socket.js`.
2. Attach the Redis adapter (`@socket.io/redis-adapter`) so events reach clients on every server instance.
3. Register the auth handshake and the three event modules.
4. Export `emitTo(room, event, payload)` for services to call after database writes.

Step 10.2: Rooms

Room	Who joins	How

`business:<id>`	Owner and staff screens	Automatically after a valid JWT handshake (auth.socket.js). The business id comes from the token only.
`display:<slug>`	TV displays	Public join with a slug; only if the plan has `tvDisplay` and the business is live.
`token:<id>`	One customer's phone	Public join with token id and `guestSecret`.
`public:<slug>`	Join pages	Public; receives waiting counts and open or closed changes only.

Step 10.3: Events

Event	Sent to	Payload
`queue:updated`	business, display, public	Changed queue keys and new counts. Clients refetch or patch their list.
`token:updated`	token room, business	Token with status, ahead, wait, nowServing.
`token:called`	display, token room	Code and resource name (triggers the TV chime and flash).
`business:updated`	business, public, display	Open or closed, branding, live status.
`notification:new`	business	Admin bell notification.

When a token is called or a line moves, the position of every token behind it changes. Recompute `ahead` for the affected line and emit to each `token:<id>` room, or emit one `queue:updated` and let token pages refetch their own status.

Step 10.4: Rules

- Emit only after the database write succeeds.
- Never send customer phone numbers or names to public or display rooms; displays only need token codes.
- Clients refetch full state on reconnect, so missed events do not matter.

Done when

- ☐ Calling a token updates the customer page, the TV, and other staff screens without a refresh
- ☐ A socket with business A's token never receives business B events
- ☐ Two server instances behind a load balancer both deliver events (Redis adapter)

Phase 11 Staff, invites, and PIN switching

> **Goal:** owners invite staff by email; staff accept, log in at the same login page, and switch users with a PIN.
> **Files:** `invite.routes/controller, services/invite.service.js, validators/invite.validator.js, staff.controller.js, tests/invite.test.js`

Step 11.1: Seat counting

Seats used = active non-owner users + pending, unexpired invites. Compare with the plan's `staffSeats` limit (Free 1, Pro 3, Business 15).

Step 11.2: Invites (invite.service.js)

Endpoint	Behaviour
`GET /api/admin/staff`	Staff users plus invites, with status: Active, Invited, Expired, Revoked, and days until expiry.
`POST /api/admin/invites`	Body: email, role (staff or manager), resourceId. Reject existing accounts and duplicate pending invites; enforce seats (`seat_limit`). Generate a random token (nanoid, 32 characters), store only its SHA-256 hash, set `expiresAt = now + 7 days` , and email the link `CLIENT_URL/accept-invite/<token>` .
`POST /api/admin/invites/:id/resend`	New token and expiry; the old link stops working. Re-check seats if it had expired.
`POST /api/admin/invites/:id/revoke`	Status revoked; frees the seat.
`GET /api/invites/:token` (public)	Hash the token and look it up. Return business name, logo, role, and resource, or 410 "This invite link is no longer valid".
`POST /api/invites/:token/accept` (public)	Body: name, password, confirm, optional 4-digit PIN. In a transaction: create the user (role from the invite, linked business and resource, hashed password and PIN, random PIN if none), mark the invite accepted. Log the user in.

Until an email provider is chosen, return the invite link in the API response in development only, the same way the frontend shows a "mock email preview".

Step 11.3: Managing staff

`PATCH /api/admin/staff/:id` changes resourceId or role. `POST /api/admin/staff/:id/reset-pin` returns a new random PIN once. `POST /api/admin/staff/:id/deactivate` sets status deactivated and bumps `refreshTokenVersion` (logs them out); `/reactivate` re-checks seats.

Step 11.4: PIN switching (POST /api/staff/switch-user)

1. Requires a logged-in staff or manager session; the target user must be active and in the same business.
2. Check a Redis lock `pinlock:<businessId>`; if set, return 423 `locked` with the seconds left.
3. Compare the PIN with bcrypt. On failure increment `pinfail:<businessId>` (expires in 15 minutes); at 5 failures set the lock for 30 seconds and reset the counter. Return `wrong_pin` with remaining attempts.
4. On success clear the counter and issue a new token pair for the target user.

`GET /api/staff/team` returns names and avatar colours for the switch-user grid (no emails or PINs).

Step 11.5: tests/invite.test.js

Cover: seat limit blocks the second invite on Free; accepting creates a staff user who can log in at `/api/auth/login`; used, revoked, expired, and replaced (after resend) tokens all return 410; PIN lockout after 5 failures and unlock after 30 seconds (use fake timers or a short test TTL); deactivated staff cannot log in.

Done when

- ☐ The full invite flow works with a real email (or the dev-mode link)
- ☐ Staff only ever see their own business and assigned resource
- ☐ `tests/invite.test.js` passes

Phase 12 Go live, public pages, and TV display

> **Goal:** businesses go live through a checklist; customers and TVs get everything they need by slug.
> **Files:** `middleware/goLive.middleware.js, business.controller.js, display.routes/controller, token.routes.js`

Step 12.1: Go-live checklist

`GET /api/business/go-live` returns four items, each with `done`: at least one service, at least one active resource, at least one open day in hours, slug confirmed. `POST /api/business/go-live` sets `isLive` and `isOpen` to true only when all four are done, logs activity, and emits `business:updated`.

`PATCH /api/business/open` with `{ isOpen }` is the "Pause queue (close for now)" switch.

Step 12.2: goLive.middleware.js

For public join routes: return 409 `not_live` before go live and `closed` when paused. Viewing an existing token stays allowed, so people already waiting keep seeing their status.

Step 12.3: Public business endpoint (GET /api/public/:slug)

Return only what the join page needs: name, logo, brand colour, welcome message, open, live, today's hours, labels, queue mode, active services and resources (with wait tags), customer fields, whether priority and "any available" are offered, whether "Powered by QueueLess" shows, and the current waiting count and estimate. Never return contact details, plan internals, or staff data.

Step 12.4: Display endpoint (GET /api/display/:slug)

Return branding, active resources each with their current token code and waiting count, the next three token codes, and total waiting. If the plan lacks `tvDisplay`, return 402 `PLAN_REQUIRED` so the TV shows its locked message. Codes only, no customer names.

Step 12.5: QR codes

The frontend already draws QR codes in the browser, so the backend does not need to. Optionally add `GET /api/business/qr.png` using the `qrcode` package for emailing posters, and mark `setup.qrPrinted` when it is downloaded.

Done when

- ☐ Public pages show "not open yet" before go live and "Closed" when paused
- ☐ The display endpoint is locked on Free and works on Pro
- ☐ No public response contains phone numbers or staff emails

Phase 13 Plans, billing, and payments

> **Goal:** real subscriptions with trials, upgrades, downgrades, and invoices, while plans only change limits and extras.
> **Files:** `billing.routes/controller, services/billing.service.js, validators/billing.validator.js, models/Subscription.js, Invoice.js`

Step 13.1: Plans as the single source of truth

Plan	Price (INR a month)	Counters	Tokens a day	Staff seats	Extras
Free	0	1	50	1	None
Pro	999 (14-day trial)	5	Unlimited	3	TV display, browser alerts, 100 SMS, remove branding
Business	2,999 (14-day trial)	25	Unlimited	15	Pro plus priority queues, advanced analytics, CSV export, API access, 5 locations

Enterprise	Custom	Unlimited	Unlimited	Unlimited	Everything plus custom domain and SSO

Yearly billing is 20% cheaper. Write a `hasFeature(business, key)` helper and a `requireFeature('csvExport')` middleware that returns 402 `PLAN_REQUIRED`.

Step 13.2: Endpoints

- `GET /api/billing`: current plan, cycle, usage meters (tokens today, counters, staff seats, SMS this month), payment method summary, invoices.
- `POST /api/billing/plan` with `free`: downgrade or "Skip for now". Cancels any paid subscription at period end. Deletes nothing.
- `POST /api/billing/checkout` with `{ plan, cycle }`: creates a Razorpay subscription (or Stripe Checkout session) and returns what the frontend needs to open the provider's checkout. The frontend's test-mode card form is replaced by the provider's hosted form.
- `POST /api/billing/webhook`: raw body, verify the provider signature, then handle activated, charged, failed, and cancelled events idempotently (store processed event ids). Update Subscription and Business.plan, create Invoice records, and emit `business:updated` so the UI unlocks immediately.

> Only the webhook changes the plan after a payment. Never trust a "payment succeeded" call from the browser.

Step 13.3: Downgrades and limits

When a business moves to a smaller plan, keep every counter, staff member, and token. Extras lock and new additions are blocked until usage fits, exactly as the frontend shows "over limit" meters.

Done when

- ☐ A test-mode payment upgrades the business through the webhook and unlocks the TV display
- ☐ Replaying the same webhook does not create a second invoice
- ☐ Downgrading removes no data

Phase 14 Notifications, analytics, and background jobs

> **Goal:** alerts reach customers, owners see their numbers, and time-based rules run on their own.
> **Files:** `services/notification.service.js, analytics.service.js, jobs/noShow.job.js, cleanup.job.js, reminder.job.js`

Step 14.1: notification.service.js

- **Almost your turn:** when a token's `ahead` drops to 3 or fewer, notify once. Always emit to the token room (the page shows the banner and vibrates). If the business has `notifications.almostSms` and the plan has `smsAlerts` with credits left, send an SMS and increment `smsUsed`.
- **Called:** emit to the token room; optional SMS on paid plans.
- Optional Web Push (VAPID keys and a service worker) so alerts arrive with the browser in the background.
- Admin notifications (line getting long, invite expiring, SMS credits low) are stored and emitted as `notification:new`.
- Wrap SMS and email providers behind small adapter functions so they can be swapped, and log every send.

Step 14.2: analytics.service.js

Use MongoDB aggregation pipelines on Token, always matching `businessId` first:

- Today metrics: tokens issued, served, average wait (calledAt minus createdAt), no-shows, each with change against the same time yesterday.
- Daily volume and no-show trend for 7, 14, or 30 days.
- Customers per hour and average wait by hour.
- Service split (donut) and resource performance: served, average wait, average service time, no-shows, rating.

Endpoints: `GET /api/admin/analytics?range=14` (advanced parts require `advancedAnalytics`), `GET /api/admin/customers` (search, filters, pagination), and `GET /api/admin/customers/export.csv` (requires `csvExport`, streamed). For large businesses, add a nightly job that saves daily summaries to a small collection so charts stay fast.

Step 14.3: Jobs

Job	Schedule	What it does
`noShow.job.js`	Delayed job per called token (BullMQ), or every 30 seconds	If a token is still CALLED after the business's `noShowMinutes`, mark it NO_SHOW (optional per business; staff can also do it by hand) and emit updates.
`reminder.job.js`	Every minute and daily at 08:00	Notifications for invites expiring within 24 hours and the owner's daily summary when enabled.

`cleanup.job.js`	Daily at 03:00	Cancel tokens left active from previous days, mark expired invites, remove old activity logs, and anonymise customer phone numbers older than the retention period.

Run workers in the same process at first; move them to a separate worker process when traffic grows.

Done when

- ☐ A customer gets the "almost your turn" alert exactly once
- ☐ Analytics numbers match a hand count on seeded data
- ☐ Yesterday's leftover WAITING tokens are cancelled overnight

Phase 15 Security, seed data, and testing

> **Goal:** the API is safe to expose, easy to demo, and fully covered by tests.
> **Files:** `middleware/rateLimit.middleware.js, enterprise.routes/controller, src/seed/seed.js, demoData.js, tests/*.test.js`

Step 15.1: rateLimit.middleware.js

Route group	Suggested limit
Login, forgot password	10 per 15 minutes per IP and per email
Public join	5 per 10 minutes per IP; 3 per hour per phone per business
Invite accept, slug check	30 per minute per IP
Enterprise lead form	3 per hour per IP
Everything else	300 per minute per user

Store counters in Redis (`rate-limit-redis`) so limits hold across instances.

Step 15.2: Security checklist

- helmet, strict CORS (only `CLIENT_URL`), and HTTPS everywhere in production.
- Validate every input with Zod; reject unknown fields on update routes.
- Prevent NoSQL injection by never passing raw request objects into queries (Zod parsing already strips operators).
- bcrypt for passwords and PINs; hashed invite and reset tokens.
- Audit log entries for plan changes, slug changes, staff changes, and deletions (ActivityLog).
- Customer data: store only what the join form asks for, and delete or anonymise it on a schedule.

- Danger zone: `DELETE /api/business` requires the slug typed in the body, cancels the subscription, and deletes the business's data in the background.

Step 15.3: Enterprise leads

`POST /api/enterprise/leads`: validate (name, work email that is not a free mailbox, company, locations at least 1, message up to 1000 characters), save an EnterpriseLead, and email the sales inbox.

Step 15.4: Seed data (src/seed/)

Port the demo data from `frontend/src/data/` into `demoData.js`: City Care Clinic, United Bank, Campus Canteen (with the `college-canteen` redirect), Glow Studio (not live), the demo users (password `demo1234`, staff PINs), invites in every state, invoices, and 60 to 100 tokens per live business over 30 days. `seed.js` clears the development database, inserts everything with hashed secrets, and exits. Refuse to run when `NODE_ENV=production`.

Step 15.5: The six test suites

File	Built in	Covers
`auth.test.js`	Phase 5	Signup, login, refresh, logout, roles
`tenantIsolation.test.js`	Phase 6	No cross-business reads, writes, or socket events
`slug.test.js`	Phase 7	Rules, availability, history, redirects
`token.test.js`	Phase 9	Joining, numbering, limits, duplicates
`queue.test.js`	Phase 9	Ordering, modes, concurrency, transitions, ETA
`invite.test.js`	Phase 11	Seats, links, resend, revoke, PIN lockout

Use `mongodb-memory-server` (as a replica set for transactions) and a Redis mock or a test Redis database. Each suite seeds its own data. Run all suites in CI on every push.

Done when

- ☐ `npm run seed` creates the demo data and the demo logins work
- ☐ `npm test` passes all six suites
- ☐ Rate limits return 429 with a `Retry-After` header

Phase 16 Connecting the frontend and deploying

> **Goal:** the React app talks to the real API, and both run in production.

> The current frontend deliberately makes no network calls. This phase is the point where that changes, so do it as a separate, planned task once the backend passes all tests.

Step 16.1: Replace the fake backend, one area at a time

1. Add `VITE_API_URL` to the frontend and create a small API client (fetch wrapper) that adds the access token, refreshes it on 401, and turns the error shape from Phase 3 into the same `{ ok, error }` results the pages already handle.
2. Keep the function names from `fakeBackend.js` and change their bodies to API calls. Pages and components then need few or no changes.
3. Replace the Zustand persisted "database" with server data: load lists from the API and keep stores for UI state and caches only.
4. Replace the `storage`-event tab sync and the simulation with a Socket.IO client that joins the rooms from Phase 10 and updates the stores on events.
5. Switch area by area in this order, testing each: auth, onboarding, settings, services and resources, public join and token pages, staff console, TV display, invites, billing, analytics.
6. Logos: upload files to `/api/business/logo` instead of keeping data URLs.

Step 16.2: Production setup

Piece	Suggestion
API hosting	Render, Railway, Fly.io, or AWS (ECS or Elastic Beanstalk) running `npm start`, at least two instances
Database	MongoDB Atlas (replica set, daily backups, region in India for data residency)
Redis	Upstash, Redis Cloud, or ElastiCache
Frontend	Vercel, Netlify, or S3 with CloudFront; rewrite all paths to `index.html` for client-side routes
Load balancer	Enable WebSockets; sticky sessions are not needed with the Redis adapter
Monitoring	Health check on `/api/health`, structured logs, error tracking (for example Sentry), uptime alerts
CI/CD	Lint and test on every push; deploy main after tests pass; run index builds before releasing new queries

Step 16.3: Enterprise extras (later)

Custom domains (map a host header to a business; verify DNS with a TXT record), SSO (SAML or OpenID Connect), multiple locations per organisation, a public API with API keys for Business plans, and an uptime SLA.

Done when

- ☐ The full journey works end to end against the deployed API: sign up, go live, a customer joins from a phone, staff call them, the TV updates, the owner upgrades
- ☐ Two browsers on different devices stay in sync through Socket.IO
- ☐ Backups, monitoring, and alerts are in place

Appendix

Endpoint list, environment variables, final checklist

A.1 Endpoints by route file

Route file	Endpoints	Access
auth.routes.js	POST signup, login, refresh, logout, forgot-password, reset-password; GET me	Public (me: logged in)
onboarding.routes.js	POST welcome	Owner
business.routes.js	GET /; PATCH settings, branding, slug, open; GET slug/check; POST/DELETE logo; GET/POST go-live; DELETE /	Owner
admin.routes.js	Services and resources CRUD and bulk; GET customers, customers/export.csv, analytics, activity, notifications	Owner
invite.routes.js	GET /admin/staff; POST /admin/invites, :id/resend, :id/revoke; PATCH /admin/staff/:id, reset-pin, deactivate, reactivate; GET /invites/:token; POST /invites/:token/accept	Owner; accept is public
staff.routes.js	GET console, team; POST resources/:id/call-next, tokens/:id/call, arrived, done, no-show, recall, skip, remove; POST walk-in, switch-user; PATCH resources/:id/pause	Staff, manager
token.routes.js	GET /public/:slug; POST /public/:slug/tokens; GET /public/:slug/tokens/:id; POST tokens/:id/leave, rate	Public by slug
display.routes.js	GET /display/:slug	Public by slug, paid plans
billing.routes.js	GET /; POST plan, checkout; POST webhook (raw body, signature)	Owner; webhook from provider

enterprise.routes.js	POST leads	Public, rate limited

A.2 Which file does what

config/

env.js validates settings; db.js and redis.js connect; socket.js holds Socket.IO options.

models/

12 Mongoose schemas. Business-owned models always have an indexed `businessId`.

routes/ and controllers/

Routes list middleware; controllers translate HTTP to service calls and back.

services/

All business rules. Callable from controllers, jobs, sockets, and tests.

middleware/

auth, role, tenant, planLimit, goLive, validate, rateLimit, error.

sockets/ and jobs/

Real-time rooms and events; no-show, cleanup, and reminder schedules.

validators/ and utils/

Zod schemas; ApiError, asyncHandler, slug helpers, token numbering, constants.

seed/ and tests/

Demo data for development; six test suites run in CI.

A.3 Build order at a glance

Phase	Depends on	Result
1 to 3	Nothing	Server boots with consistent errors
4 to 6	1 to 3	Models, login, and tenant isolation
7 to 8	4 to 6	Owners can configure their business
9 to 10	7 to 8	Live queues with real-time updates
11 to 12	9 to 10	Staff onboarding, go live, TV display
13 to 14	11 to 12	Payments, alerts, analytics, jobs
15 to 16	All	Hardened, seeded, tested, connected, deployed

A.4 Final checklist

- ☐ Every function in `frontend/src/lib/fakeBackend.js` has a matching endpoint or service
- ☐ All admin and staff queries are filtered by the session's `businessId`
- ☐ Customers and TVs are scoped by slug only and never see private data
- ☐ All customisation works on the Free plan; limits block additions but never delete data
- ☐ Token numbers are unique under concurrent joins; two staff never call the same customer
- ☐ Old slugs redirect for 90 days
- ☐ Invites expire after 7 days, are single-use, and resending invalidates the old link
- ☐ PIN switching locks for 30 seconds after 5 wrong attempts
- ☐ Plan changes come only from verified payment webhooks
- ☐ All six test suites pass in CI
- ☐ Production has backups, monitoring, rate limits, and HTTPS